TÉCNICO
UNIVERSIDADE
DE LISBOA

Traffic Engineering

METI

Lab 3 Report [EN]

Authors:

José Oliveira (103771) jose.novais.oliveira@tecnico.ulisboa.pt
Tiago Videira (116069) tiago.g.videira@tecnico.ulisboa.pt

Group 5

2025/2026 – 2nd Semester, P4

Contents

1 Linux Network Namespaces

1.1 Exercise 4.1 – Two-Node Network

The objective of this first exercise was to understand the operation of Linux network namespaces and virtual Ethernet interfaces by creating a simple two-node topology.

A new namespace named `examplens` was created using:

```
ip netns add examplens
```

Network namespaces provide isolated networking environments inside the Linux kernel. Each namespace has its own interfaces, routing table, ARP table, and network stack, behaving similarly to an independent host.

The created namespace was verified using:

```
ip netns list
```

```
root@lima-default:/home/tiagovideira.guest# ip netns exec examplens ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
       valid_lft forever preferred_lft forever
14: veth1@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 9a:a8:f1:0c:5b:df brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.2/24 scope global veth1
       valid_lft forever preferred_lft forever
    inet6 fe80::98a8:f1ff:fe0c:5bdf/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 1: Verification of the created Linux namespace

To interconnect the default namespace with the newly created namespace, a virtual Ethernet pair (veth pair) was created:

```
ip link add veth0 type veth peer name veth1
```

A veth pair acts as a virtual cable between two interfaces. Any packet entering one side exits through the other side.

One of the interfaces was then moved into the namespace:

```
ip link set veth1 netns examplens
```

At this stage:

- `veth0` remained in the default namespace;
- `veth1` became part of the `examplens` namespace.

IP addresses were assigned to both interfaces:

```
ip addr add 10.0.0.1/24 dev veth0

ip netns exec examplens \
ip addr add 10.0.0.2/24 dev veth1
```

The command `ip netns exec` allows the execution of commands inside a specific namespace.

The loopback interface inside the namespace was also enabled:

```
ip netns exec examplens ip link set lo up
```

Each namespace has its own independent loopback interface, which is initially administratively down by default.

After configuring the interfaces, the resulting addressing information was verified.

```
root@lima-default:/home/tiagovideira.guest# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 52:55:55:ef:d4:3d brd ff:ff:ff:ff:ff:ff
    altname enx525555efd43d
    inet 192.168.5.15/24 metric 200 brd 192.168.5.255 scope global dynamic eth0
       valid_lft 2189sec preferred_lft 2189sec
    inet6 fe80::5055:55ff:feef:d43d/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether ee:bc:b4:0c:d0:9e brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
9: br-1e3456f39d2b: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:12:54:2a:71:5a brd ff:ff:ff:ff:ff:ff
    inet 172.20.20.1/24 brd 172.20.20.255 scope global br-1e3456f39d2b
       valid_lft forever preferred_lft forever
    inet6 3fff:172:20:20::1/64 scope global nodad
       valid_lft forever preferred_lft forever
    inet6 fe80::12:54ff:fe2a:715a/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
10: vethf11dbc7@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-1e3456f39d2b state UP group default
    link/ether a2:4c:f8:cb:a0:3a brd ff:ff:ff:ff:ff:ff link-netns clab-mini-leaf1
    inet6 fe80::a04c:f8ff:fecb:a03a/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
13: vethd77dd6e@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-1e3456f39d2b state UP group default
    link/ether 16:6b:33:a7:52:18 brd ff:ff:ff:ff:ff:ff link-netns clab-mini-spine1
    inet6 fe80::146b:33ff:fea7:5218/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
15: veth0@if14: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 66:e5:1e:f3:e5:d3 brd ff:ff:ff:ff:ff:ff link-netns examplens
    inet 10.0.0.1/24 scope global veth0
       valid_lft forever preferred_lft forever
    inet6 fe80::64e5:1eff:fef3:e5d3/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 2: IP addressing configuration inside the default namespace

The namespace addressing was also validated:

```
root@lima-default:/home/tiagovideira.guest# ip netns exec examplens ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
       valid_lft forever preferred_lft forever
24: veth1@if25: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 9a:a8:f1:0c:5b:df brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.2/24 scope global veth1
       valid_lft forever preferred_lft forever
    inet6 fe80::98a8:f1ff:fe0c:5bdf/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 3: IP addressing configuration inside the `examplens` namespace

Connectivity tests were then performed using ICMP echo requests.

From the default namespace:

```
ping 10.0.0.2
```

And from inside the namespace:

```
ip netns exec examplens ping 10.0.0.1
```

```
root@lima-default:/home/tiagovideira.guest# ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.121 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.189 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.194 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.172 ms
^C
--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4121ms
rtt min/avg/max/mdev = 0.121/0.164/0.194/0.027 ms
root@lima-default:/home/tiagovideira.guest# ip netns exec examplens ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.100 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.213 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.163 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.152 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.212 ms
^C
--- 10.0.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4089ms
rtt min/avg/max/mdev = 0.100/0.168/0.213/0.042 ms
```

Figure 4: Bidirectional connectivity validation between namespaces

The successful bidirectional communication demonstrated:

- correct namespace creation;
- successful virtual interface configuration;
- proper operation of veth pairs;
- isolated network stack functionality inside Linux namespaces.

This exercise introduced the basic concepts of Linux network virtualization and namespace-based network isolation.

1.2 Exercise 4.2 – Three-Node Network

The objective of this exercise was to emulate a routed topology using Linux network namespaces and virtual Ethernet interfaces.

Two namespaces were created:

- `near`
- `far`

The resulting topology consisted of three logical nodes:

$$Host \leftrightarrow near \leftrightarrow far$$

The namespaces were verified using:

```
ip netns list
```

```
root@lima-default:/home/tiagovideira.guest# ip netns list
far (id: 3)
near (id: 2)
```

Figure 5: Namespaces created for the routed topology

Two virtual Ethernet pairs were created:

- one pair connecting the host to the `near` namespace;
- another pair connecting the `near` namespace to the `far` namespace.

The interfaces were distributed among the namespaces using:

```
ip link set <interface> netns <namespace>
```

IP addresses were assigned as follows:

- Host side: `10.0.1.1/24`
- Near namespace (towards host): `10.0.1.2/24`
- Near namespace (towards far): `10.0.2.1/24`
- Far namespace: `10.0.2.2/24`

After assigning addresses, all interfaces and loopback interfaces were activated.

Since the `near` namespace interconnects two distinct IP networks, it was configured to operate as a virtual router.

IP forwarding was enabled using:

```
ip netns exec near \
sysctl -w net.ipv4.ip_forward=1
```

Static routing entries were then configured.

On the host side:

```
ip route add 10.0.2.0/24 via 10.0.1.2
```

On the `far` namespace:

```
ip netns exec far \
ip route add 10.0.1.0/24 via 10.0.2.1
```

The resulting routing tables were verified.

```
root@lima-default:/home/tiagovideira.guest# ip route
default via 192.168.5.2 dev eth0 proto dhcp src 192.168.5.15 metric 200
10.0.1.0/24 dev veth0 proto kernel scope link src 10.0.1.1
10.0.2.0/24 via 10.0.1.2 dev veth0
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.20.20.0/24 dev br-1e3456f39d2b proto kernel scope link src 172.20.20.1
192.168.5.0/24 dev eth0 proto kernel scope link src 192.168.5.15 metric 200
192.168.5.2 dev eth0 proto dhcp scope link src 192.168.5.15 metric 200
```

Figure 6: Routing table of the default namespace

```
root@lima-default:/home/tiagovideira.guest# ip netns exec near ip route
10.0.1.0/24 dev veth1 proto kernel scope link src 10.0.1.2
10.0.2.0/24 dev veth2 proto kernel scope link src 10.0.2.1
root@lima-default:/home/tiagovideira.guest# ip netns exec far ip route
10.0.1.0/24 via 10.0.2.1 dev veth3
10.0.2.0/24 dev veth3 proto kernel scope link src 10.0.2.2
```

Figure 7: Routing tables inside the `near` and `far` namespaces

Connectivity tests confirmed successful communication between all nodes.
From the default namespace:

```
ping 10.0.2.2
```

From the `far` namespace:

```
ip netns exec far ping 10.0.1.1
```

```
root@lima-default:/home/tiagovideira.guest# ping 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.299 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=63 time=0.122 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=63 time=0.203 ms
64 bytes from 10.0.2.2: icmp_seq=4 ttl=63 time=0.138 ms
64 bytes from 10.0.2.2: icmp_seq=5 ttl=63 time=0.120 ms
^C
--- 10.0.2.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4065ms
rtt min/avg/max/mdev = 0.120/0.176/0.299/0.068 ms
root@lima-default:/home/tiagovideira.guest# ip netns exec far ping 10.0.1.1
PING 10.0.1.1 (10.0.1.1) 56(84) bytes of data.
64 bytes from 10.0.1.1: icmp_seq=1 ttl=63 time=0.089 ms
64 bytes from 10.0.1.1: icmp_seq=2 ttl=63 time=0.193 ms
64 bytes from 10.0.1.1: icmp_seq=3 ttl=63 time=0.139 ms
64 bytes from 10.0.1.1: icmp_seq=4 ttl=63 time=0.226 ms
64 bytes from 10.0.1.1: icmp_seq=5 ttl=63 time=0.143 ms
^C
--- 10.0.1.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4065ms
rtt min/avg/max/mdev = 0.089/0.158/0.226/0.047 ms
```

Figure 8: Connectivity validation through the Linux virtual router

The observed TTL decrement from 64 to 63 confirmed that packets traversed the `near` namespace, validating that IP forwarding and routing were functioning correctly.

This exercise demonstrated:

- Linux namespace interconnection using veth pairs;
- static route configuration;
- IP forwarding operation;
- Linux namespaces functioning as virtual routers.

The experiment showed that Linux namespaces can emulate complete routed topologies while maintaining isolated network stacks for each virtual node.

1.3 Exercise 4.3 — Multi-Network Topology with Linux Namespaces and Bridges

In this exercise, a complete virtual network topology was implemented using Linux network namespaces, virtual Ethernet interfaces (veth pairs), Linux bridges, and static routing. The objective was to emulate multiple independent LANs interconnected through Linux routers, reproducing the topology presented in the assignment.

The topology consisted of:

- Two Linux routers: `r1` and `r2`
- Three Linux bridges acting as switches: `sw1`, `sw2`, and `sw3`
- Seven hosts distributed across three different LANs:
  - LAN1: `h1`, `h2`
  - LAN2: `h3`, `h4`
  - LAN3: `h5`, `h6`, `h7`

Each host and router was isolated inside its own network namespace. Connectivity between namespaces was established using veth pairs, while Linux bridges were used to emulate Layer 2 Ethernet switches.

The following IPv4 addressing scheme was adopted:

Network	Address Space
LAN1 (SW1)	10.0.1.0/24
R1 – R2 Backbone	10.0.2.0/24
LAN2 (SW2)	10.0.3.0/24
LAN3 (SW3)	10.0.4.0/24

Table 1: IPv4 addressing plan used in Exercise 4.3

Figure 9 shows the created network namespaces corresponding to all hosts and routers in the topology.

```
root@lima-default:/home/tiagovideira.guest# ip netns list
r2 (id: 5)
r1 (id: 4)
h7 (id: 10)
h6 (id: 9)
h5 (id: 8)
h4 (id: 7)
h3 (id: 6)
h2 (id: 3)
h1 (id: 2)
```

Figure 9: Network namespaces created for the topology

Linux bridges were then configured to emulate Ethernet switches, interconnecting the different hosts and routers belonging to the same LAN segment.

Figure 10 presents the bridge associations between interfaces and Linux bridges.

```
root@lima-default:/home/tiagovideira.guest# bridge link
10: vethf11dbc7@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br-1e3456f39d2b state forwarding priority 32 cost 2
13: vethd77dd6e@eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master br-1e3456f39d2b state forwarding priority 32 cost 2
29: sw1-h1@if30: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw1 state forwarding priority 32 cost 2
31: sw1-h2@if32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw1 state forwarding priority 32 cost 2
33: sw1-r1@if34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw1 state forwarding priority 32 cost 2
37: sw2-h3@if38: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw2 state forwarding priority 32 cost 2
39: sw2-h4@if40: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw2 state forwarding priority 32 cost 2
41: sw2-r2@if42: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw2 state forwarding priority 32 cost 2
43: sw3-h5@if44: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw3 state forwarding priority 32 cost 2
45: sw3-h6@if46: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw3 state forwarding priority 32 cost 2
47: sw3-h7@if48: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw3 state forwarding priority 32 cost 2
49: sw3-r2@if50: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master sw3 state forwarding priority 32 cost 2
```

Figure 10: Bridge interface associations in the virtual topology

Additionally, VLAN and bridge membership information was inspected using the `bridge vlan` command, confirming that all interfaces were correctly associated with their respective bridge domains.

```
root@lima-default:/home/tiagovideira.guest# bridge vlan
port              vlan-id
docker0           1 PVID Egress Untagged
br-1e3456f39d2b   1 PVID Egress Untagged
vethf11dbc7       1 PVID Egress Untagged
vethd77dd6e       1 PVID Egress Untagged
sw1               1 PVID Egress Untagged
sw2               1 PVID Egress Untagged
sw3               1 PVID Egress Untagged
sw1-h1            1 PVID Egress Untagged
sw1-h2            1 PVID Egress Untagged
sw1-r1            1 PVID Egress Untagged
sw2-h3            1 PVID Egress Untagged
sw2-h4            1 PVID Egress Untagged
sw2-r2            1 PVID Egress Untagged
sw3-h5            1 PVID Egress Untagged
sw3-h6            1 PVID Egress Untagged
sw3-h7            1 PVID Egress Untagged
sw3-r2            1 PVID Egress Untagged
```

Figure 11: Bridge VLAN and port membership information

Static routes were configured on both Linux routers in order to provide Layer 3 connectivity between the different LANs.

Figure 12 shows the routing table configured on router `r1`.

```
root@lima-default:/home/tiagovideira.guest# ip netns exec r1 ip route
10.0.1.0/24 dev r1-veth1 proto kernel scope link src 10.0.1.1
10.0.2.0/24 dev r1-veth2 proto kernel scope link src 10.0.2.1
10.0.3.0/24 via 10.0.2.2 dev r1-veth2
10.0.4.0/24 via 10.0.2.2 dev r1-veth2
```

Figure 12: Routing table configured on router r1

Figure 13 shows the routing table configured on router `r2`.

```
root@lima-default:/home/tiagovideira.guest# ip netns exec r2 ip route
10.0.1.0/24 via 10.0.2.1 dev r2-veth1
10.0.2.0/24 dev r2-veth1 proto kernel scope link src 10.0.2.2
10.0.3.0/24 dev r2-veth2 proto kernel scope link src 10.0.3.1
10.0.4.0/24 dev r2-veth3 proto kernel scope link src 10.0.4.1
```

Figure 13: Routing table configured on router r2

After completing the Layer 2 and Layer 3 configuration, end-to-end connectivity tests were performed between hosts located in different LANs.

Figure 14 demonstrates successful communication between host `h1` in LAN1 and host `h7` in LAN3.

```
root@lima-default:/home/tiagovideira.guest# ip netns exec h1 ping 10.0.4.30
PING 10.0.4.30 (10.0.4.30) 56(84) bytes of data.
64 bytes from 10.0.4.30: icmp_seq=1 ttl=62 time=0.144 ms
64 bytes from 10.0.4.30: icmp_seq=2 ttl=62 time=0.297 ms
64 bytes from 10.0.4.30: icmp_seq=3 ttl=62 time=0.221 ms
^C
--- 10.0.4.30 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2020ms
rtt min/avg/max/mdev = 0.144/0.220/0.297/0.062 ms
```

Figure 14: Successful end-to-end ICMP communication between LAN1 and LAN3

The obtained TTL value of 62 confirms that packets traversed two intermediate routers before reaching the destination.

To further validate the forwarding path across the virtual network, a traceroute operation was executed from `h1` towards `h7`.

Figure 15 clearly shows the packet traversal through routers `r1` and `r2` before reaching the final destination host.

```
root@lima-default:/home/tiagovideira.guest# ip netns exec h1 traceroute 10.0.4.30
traceroute to 10.0.4.30 (10.0.4.30), 30 hops max, 60 byte packets
 1  10.0.1.1 (10.0.1.1)  0.349 ms  0.019 ms  0.012 ms
 2  10.0.2.2 (10.0.2.2)  0.035 ms  0.012 ms  0.010 ms
 3  10.0.4.30 (10.0.4.30)  0.041 ms  0.013 ms  0.013 ms
```

Figure 15: Traceroute showing packet traversal across the virtual topology

This exercise demonstrated how Linux namespaces, virtual Ethernet interfaces, Linux bridges, and static routing can be combined to emulate realistic multi-network topologies entirely in software. The resulting topology successfully provided both Layer 2 switching and Layer 3 routing functionality between multiple isolated virtual networks.